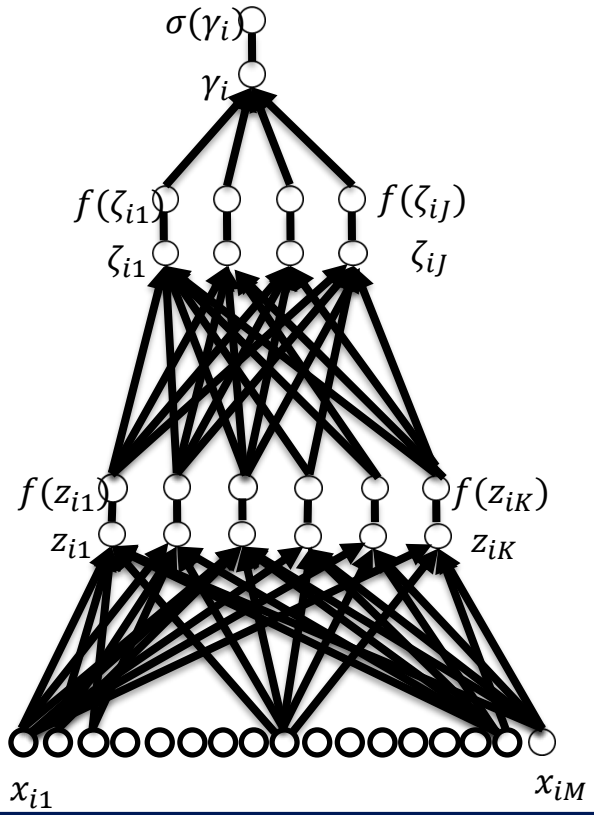
ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

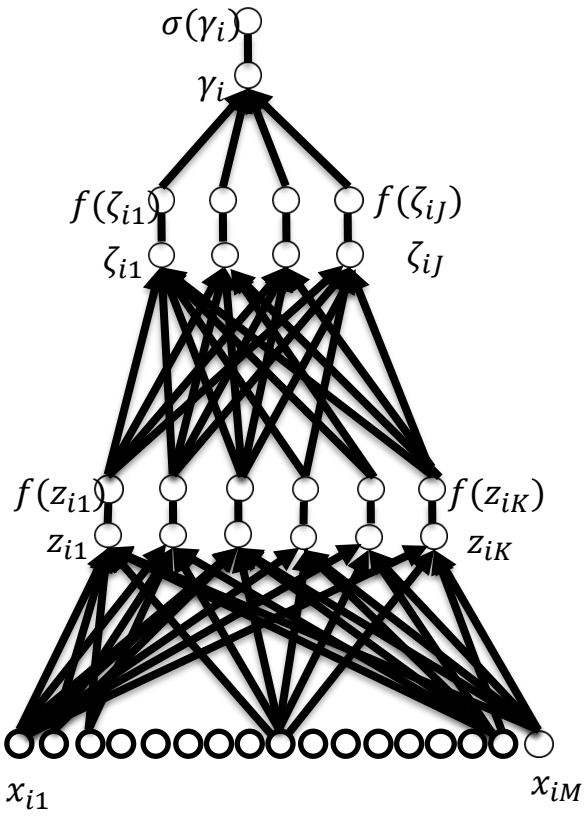
ECE 590
Neural-Network-Based Large Language Models

Lecture 4
01/20/2026

Multilayer Perceptron: Neural Network Deep Learning

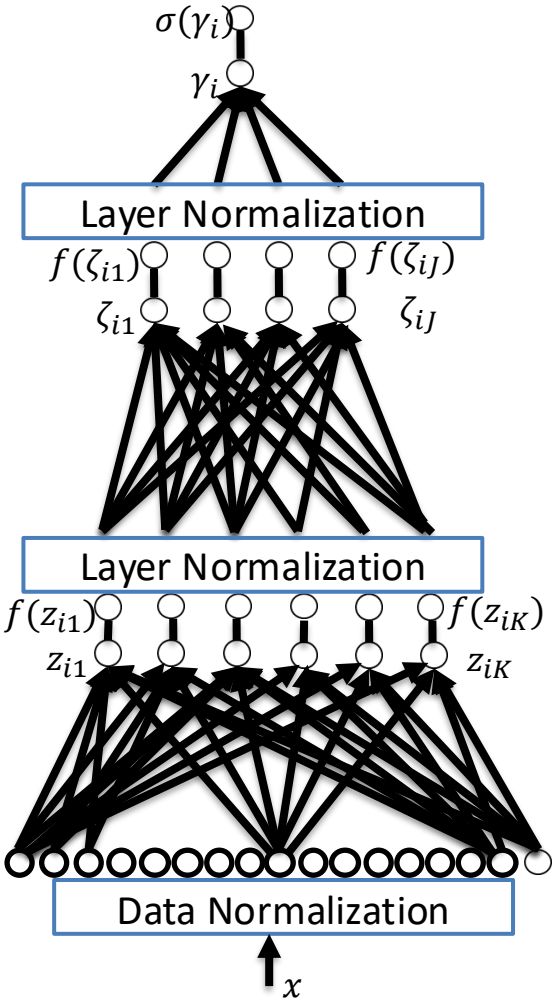


Multilayer Perceptron: Neural Network Deep Learning



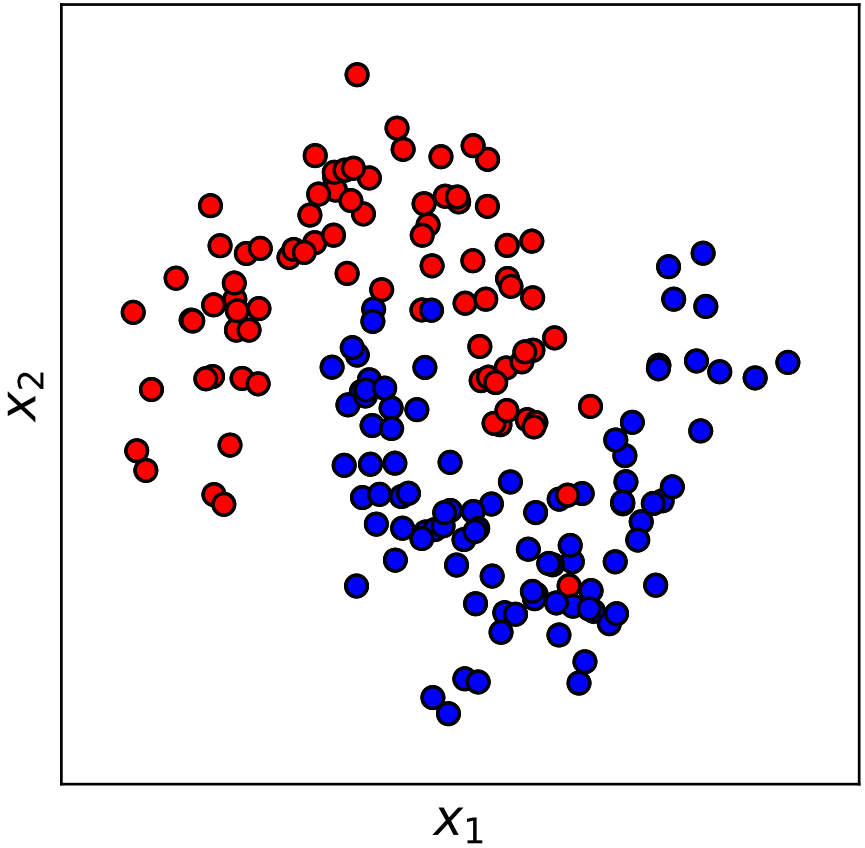
In modern NNs, based on our earlier discussions, learned features are typically normalized at each layer

Input data also normalized

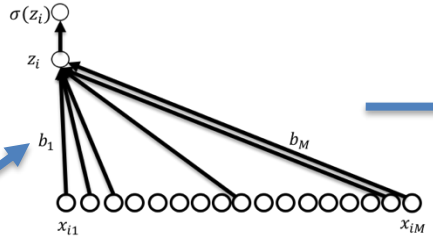

$\sigma(\gamma_i)$
γ_i
Layer Normalization
$f(\zeta_{i1})$
$f(\zeta_{iJ})$
ζ_{i1}
ζ_{iJ}
Layer Normalization
$f(z_{i1})$
$f(z_{iK})$
z_{i1}
z_{iK}
Data Normalization
x

Can learn non-linear classifications

By learning multiple layers and transformation, it is possible to have a non-linear classifier capable of more accurately capturing the data.

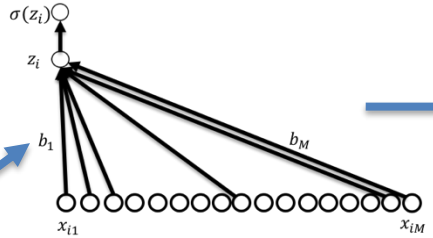


Does this work with MNIST?

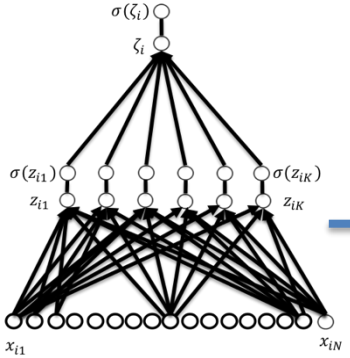


~91% Accurate

Does this work with MNIST?



~91% Accurate



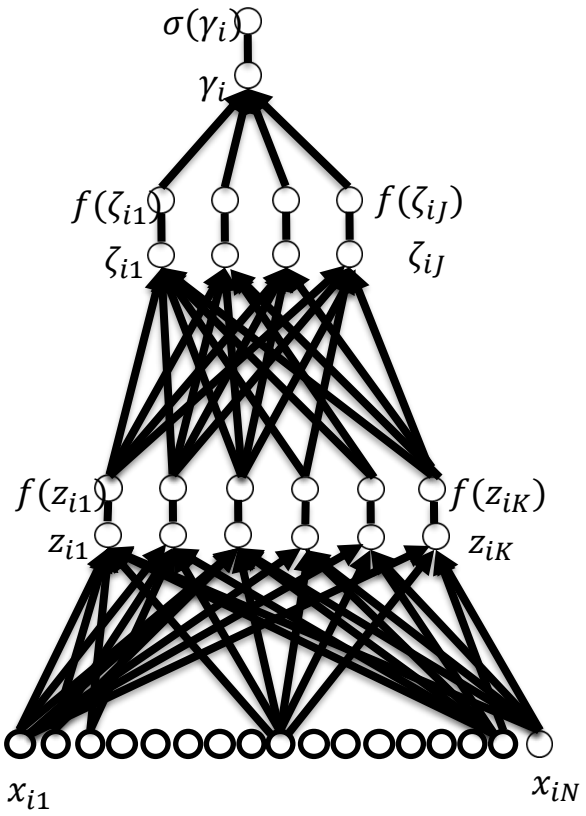
~96% Accurate

Perspective on This Model

- With logistic regression, or similar linear or generalized linear model, it is assumed data well analyzed along a line through the origin

- With an MLP, data x is transformed by a series of affine transformations followed by pointwise nonlinearities

- Critically important: At the output layer of the MLP, there is a generalized linear model, in the <u>feature space</u>

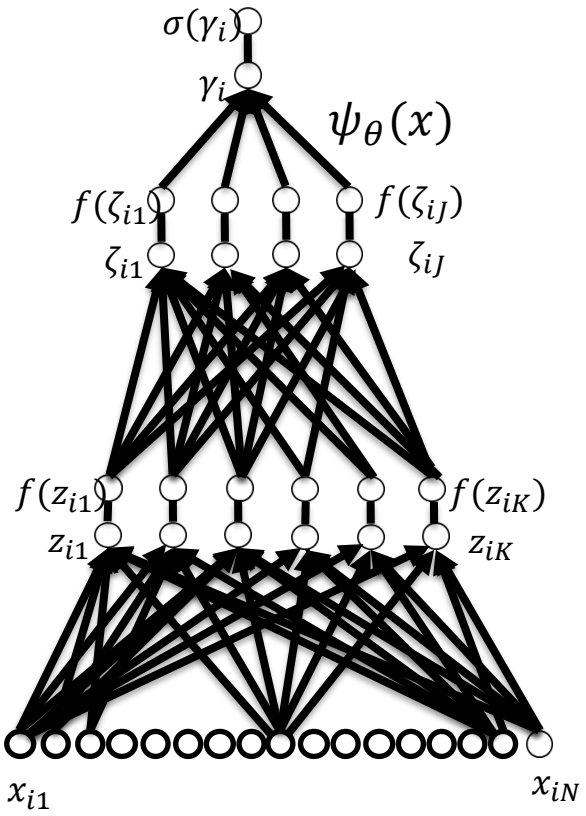


Deep Learning → Representation Learning

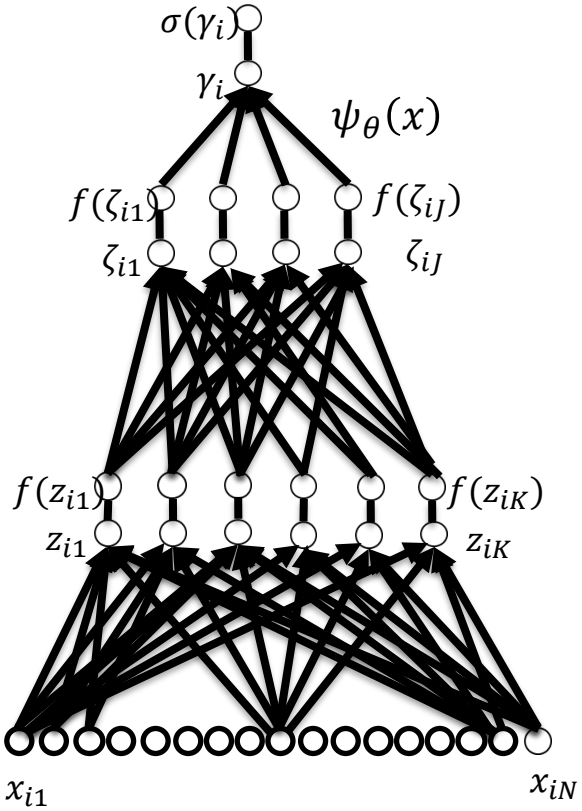
- We may see deep learning, manifested by multiple NN layers, as learning a nonlinear transformation of the data such that a linear model in the final representation is valid

- This is also called *representation learning*

- We almost always have a generalized linear model at the output layer, which is why understanding linear models well is so important

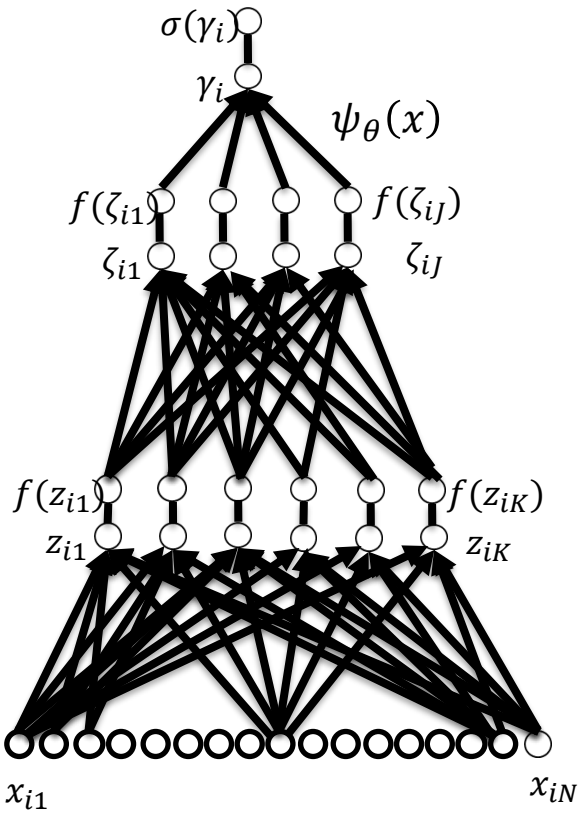


Thinking More About Representation Learning



- The input data $x \in R^N$ and features $\psi_\theta(x) \in R^J$
- In general, $N \neq J$ and possible $J > N$
- We look at this as a function $\psi_\theta(x)$ where θ are the NN model parameters (weights and biases at each layer)
- Function $\psi_\theta(x)$ represents the nonlinear portion of the NN

Thinking More About Representation Learning



- The output of linear layer, before nonlinear $\sigma(\cdot)$, is

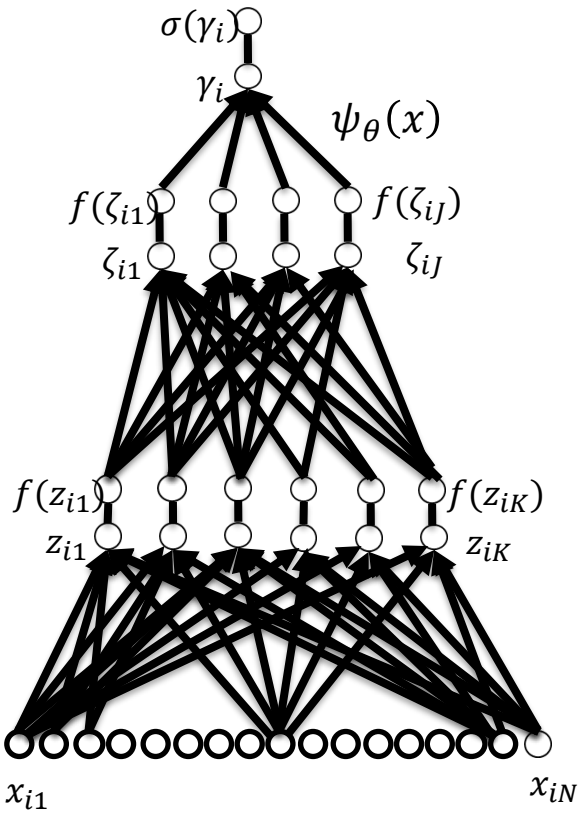
$$\gamma = w \cdot \psi_\theta(x) + b$$

- Vector $w \in R^J$ resides in feature space

- Because of the properties of the inner product, w must reside within a linear subspace spanned by $\{\psi_\theta(x_i)\}_{i=1,M}$

- Assume that the training data composed of $\{(x_i, y_i)\}_{i=1,M}$

Thinking More About Representation Learning



- The output of linear layer, before nonlinear $\sigma(\cdot)$, is

$$\gamma = w \cdot \psi_\theta(x) + b$$

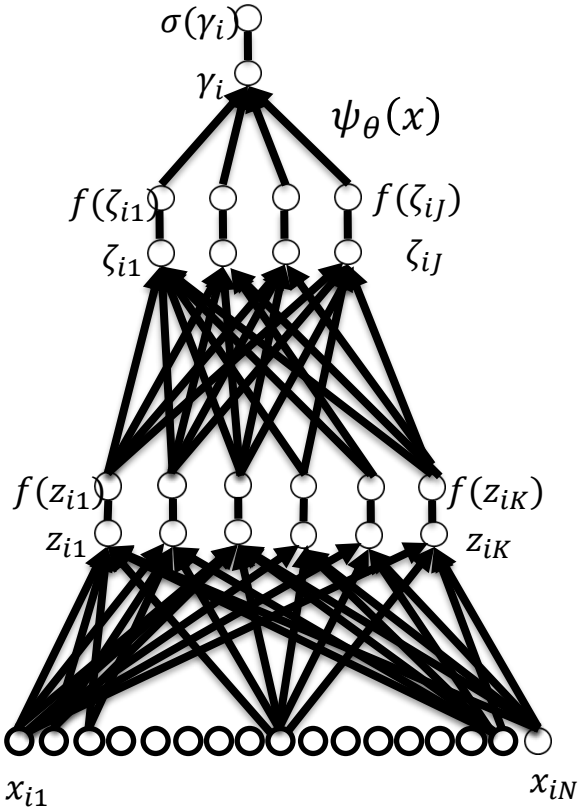
- Assume that the training data composed of $\{(x_i, y_i)\}_{i=1,M}$

- This means we anticipate that w may be expressed as

$$w = \sum_{i=1}^{M} a_i \psi_\theta(x_i)$$

Kernel Learning

$$p(Y = 1|X = x) = \sigma[\gamma(x)]$$



- The output of linear layer, before nonlinear $\sigma(\cdot)$, is

$$\gamma = w \cdot \psi_\theta(x) + b$$

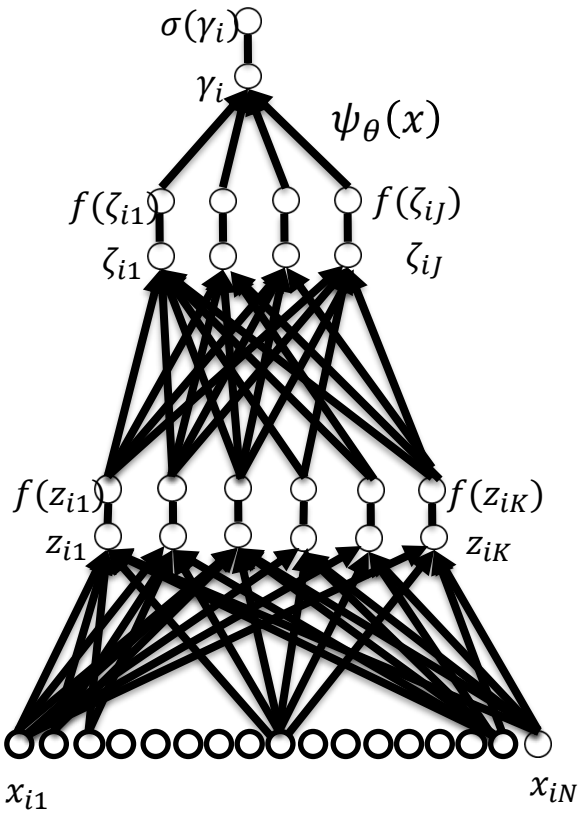
- Assume that the training data composed of $\{(x_i, y_i)\}_{i=1,M}$

$$w = \sum_{i=1}^{M} a_i \psi_\theta(x_i)$$

- Consequently:

$$\gamma(x) = \sum_{i=1}^{M} a_i \psi_\theta(x_i) \cdot \psi_\theta(x) + b = \sum_{i=1}^{M} a_i k_\theta(x, x_i) + b$$

Putting it All Together



- Logistic regression: Generalized linear model in data space:

$$\sigma(x \cdot w + b)$$

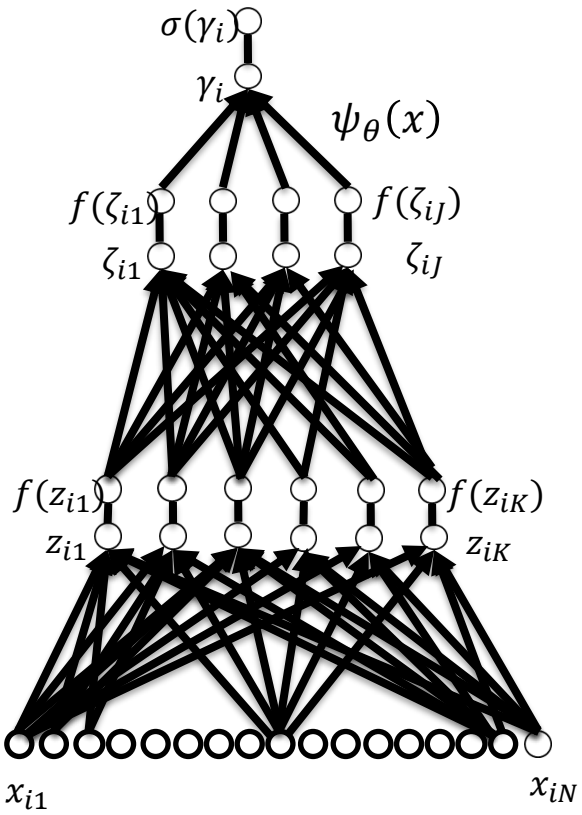
- Neural network: Generalized linear model in feature space:

$$\sigma[\psi_\theta(x) \cdot w + b]$$

- From perspective of kernel learning, the latter is

$$\sigma\{\sum_{i=1}^{M} a_i k_\theta(x, x_i) + b\}$$

Putting it All Together



- Logistic regression: Generalized linear model in data space:

$$\sigma(x \cdot w + b)$$

Linear from the perspective of x

- Neural network: Generalized linear model in feature space

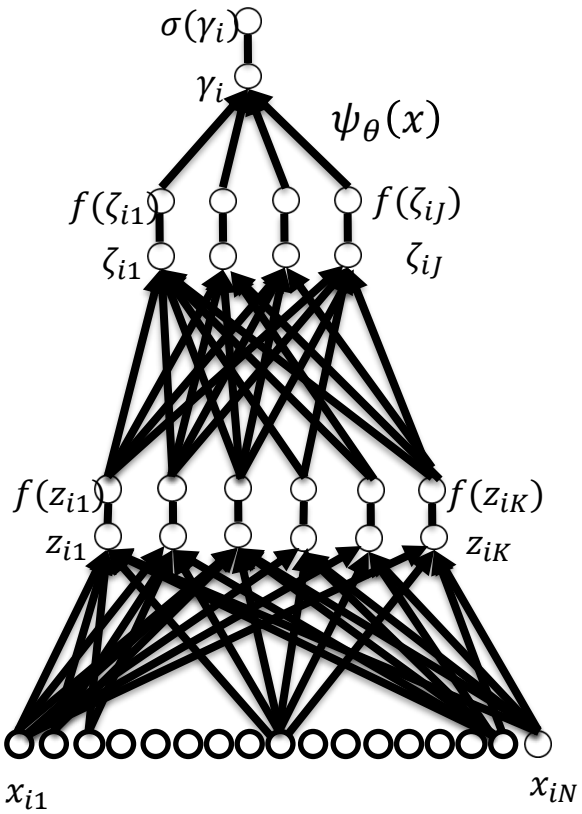
$$\sigma[\psi_\theta(x) \cdot w + b]$$

- From perspective of kernel learning, the latter is

$$\sigma\{\sum_{i=1}^{M} a_i k_\theta(x, x_i) + b\}$$

Nonlinear from the perspective of x

Putting it All Together



- Logistic regression: Generalized linear model in data space:

$$\sigma(x \cdot w + b)$$

Linear from the perspective of x

- Neural network: Generalized linear model in feature space

$$\sigma[\psi_\theta(x) \cdot w + b]$$

Linear in feature space $\psi_\theta(x)$

- From perspective of kernel learning, the latter is

$$\sigma\{\sum_{i=1}^{M} a_i k_\theta(x, x_i) + b\}$$

Nonlinear from the perspective of x

Historical Footnote: Kernel Machines

- Until relatively recently, there was not sufficient data and computation to reliably <u>learn</u> kernels like

$$k_\theta(x, x_i) = \psi_\theta(x) \cdot \psi_\theta(x_i)$$

- Therefore, in practice kernels were "handcrafted", with a few parameters, e.g., the radial basis function

$$k_\lambda(x, x') = \exp(-\lambda \|x - x'\|_2)$$

- Much smaller number of parameters to learn, here just one, λ

$$\sigma\{\sum_{i=1}^{M} a_i k_\lambda(x, x_i) + b\}$$

Historical Footnote: Kernel Machines

- Until relatively recently, there was not sufficient data and computation to reliably <u>learn</u> kernels like $k(x, x_i) = \psi_\theta(x) \cdot \psi_\theta(x_i)$

- Therefore, in practice kernels were "handcrafted", with a few parameters, e.g., the radial basis function

$$k_\lambda(x, x') = \exp(-\lambda \|x - x'\|_2)$$

- Much smaller number of parameters to learn, here just one, λ

$$\sigma\{\sum_{i=1}^{M} a_i k_\lambda(x, x_i) + b\}$$

Widely used example: Support Vector Machine (SVM), in which the set of weights $\{a_i\}_{i=1,M}$ is *sparse*

Transformers as Learned Kernel Machines

- Very difficult to "handcraft" kernels for sophisticated data, like images, sound and natural language (*e.g.*, Euclidean distance is not a good metric of closeness)

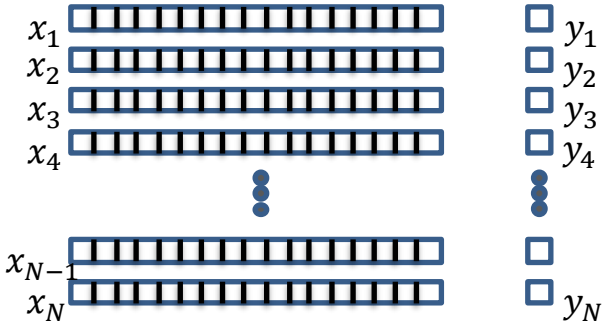
- Deep learning and NNs may be framed as kernel learning for general data

- The Transformer network, which is the focus of this class, is a very powerful form of nonlinear feature learning, particularly for natural language, images, and related data

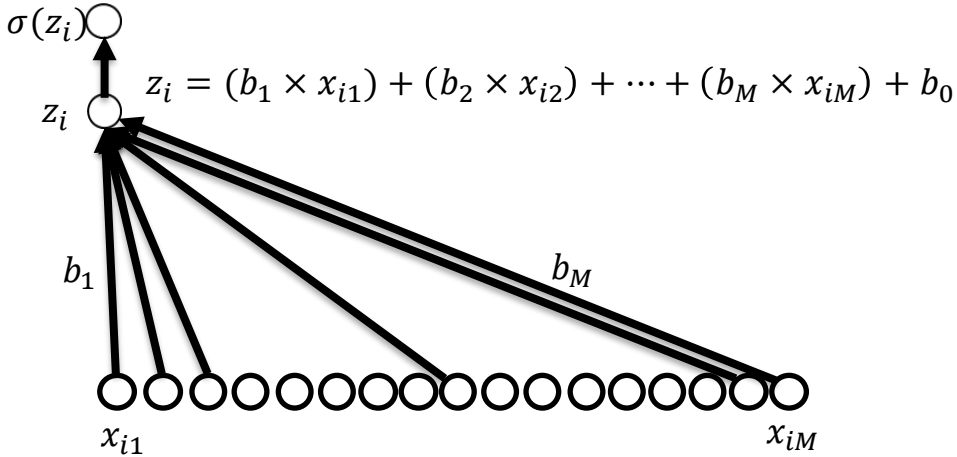
- The output layer of a Transformer is linear, or generalized linear, and therefore like virtually all NNs, it may be framed as representation learning, or equivalently kernel learning

How do we learn the parameters of a Deep Model?

(Recall) Learning Model Parameters



Training Set



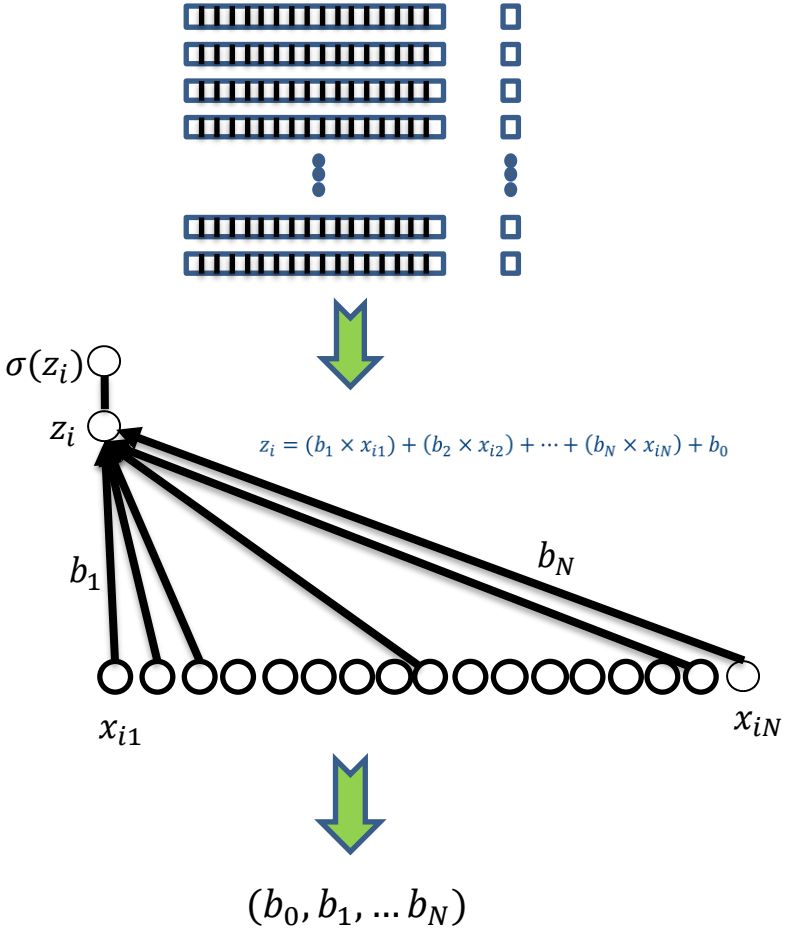
Logistic Regression Model (or "Network")

How do we actually do this?

Learned Parameters $(b_0, b_1, \ldots b_M)$

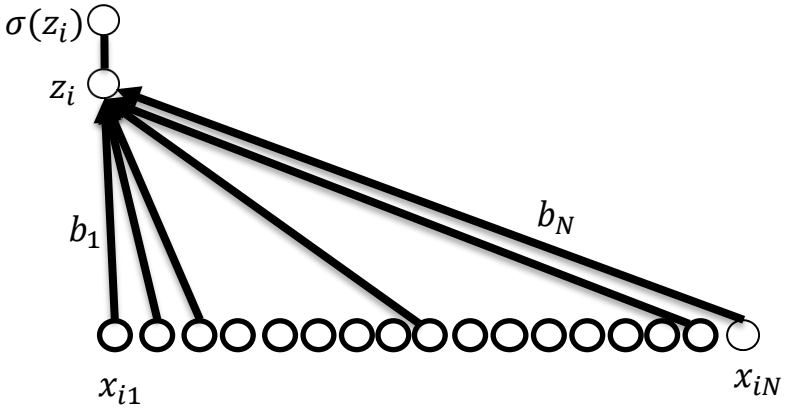
Fundamental Question:

- Given a large amount of data and a model/network to fit, how to *efficiently* and *effectively* learn the model parameters?



What are we trying to do?

- Want to learn parameters that give us the best performance
- Essentially: given data, find the "best" $\boldsymbol{b}$ for that data
- How do we define performance?



“Empirical Risk Minimization”

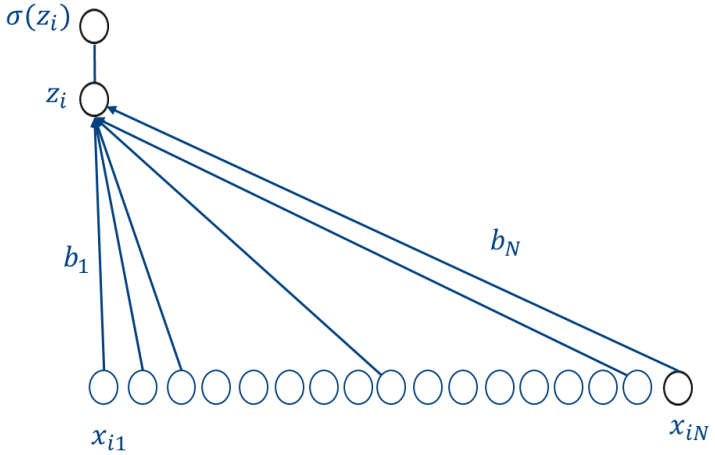
- A *loss function* $\ell(true, prediction)$ defines a penalty for poor predictions
- Want to minimize average loss
- Mathematically, this can be stated as

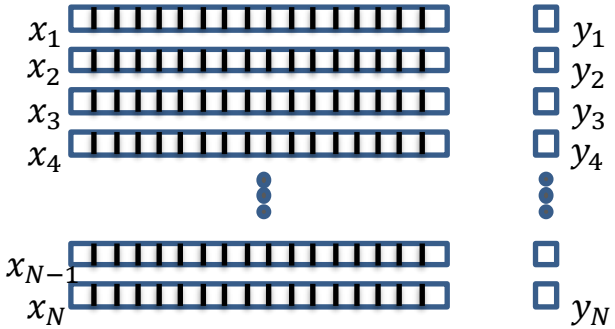
$$\boldsymbol{b}^* = \arg\min_{\boldsymbol{b}} \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, \sigma(z_i))$$

Loss function

Guess

True Label





What is the Loss Function?

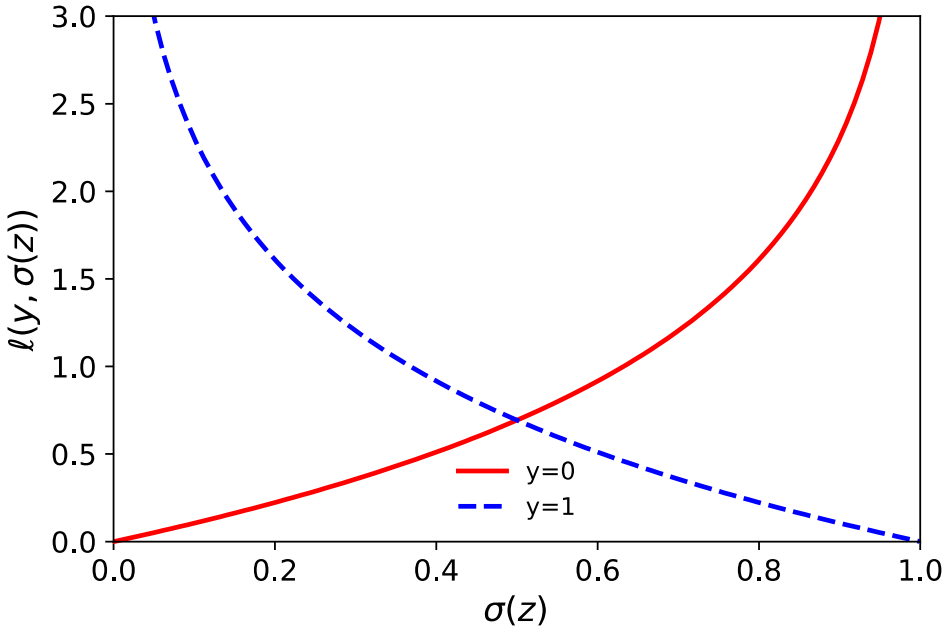
- Define $\sigma(z_i)$ as the predicted probability
- y_i is our true label
- Can be viewed as the negative log-likelihood
  $\ell\big(y_i, \sigma(z_i)\big) = -\log p(y_i | \sigma(z_i))$
- Specific mathematical form is:
  $\ell\big(y, \sigma(z)\big) = -y \log \sigma(z) - (1 - y) \log\big(1 - \sigma(z)\big)$

Visualization of Logistics Loss Function

- The logistic/cross-entropy loss is:

$$\ell(y, \sigma(z)) = -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z))$$

- Dashed blue line shows loss when the true prediction is positive
- When we give 100% of a 1, we pay no penalty
- If we are less confident or wrong, we pay an increasing penalty



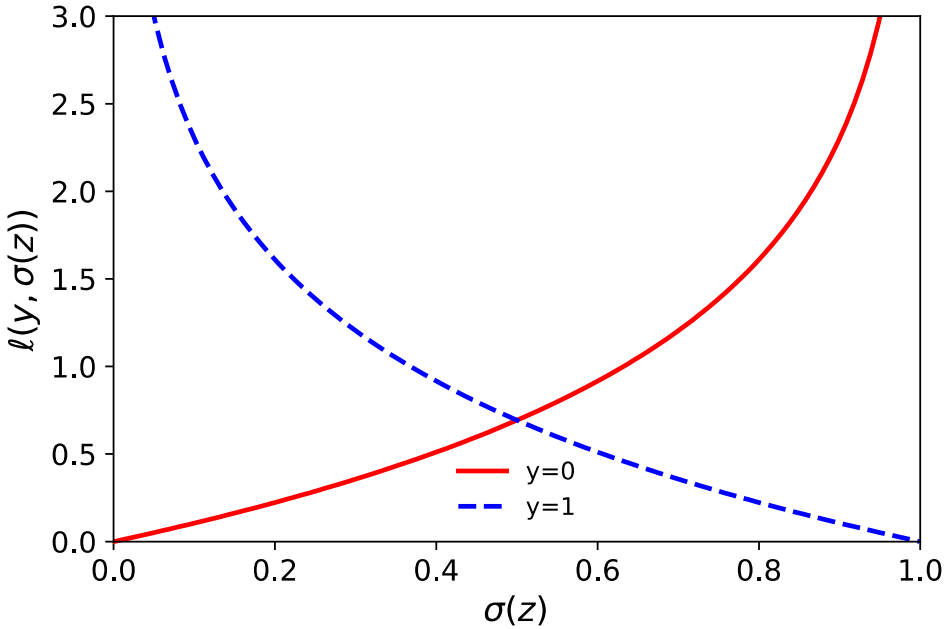
Predicted Probability of a "one"

Visualization of Logistics Loss Function

- The logistic/cross-entropy loss is:

$$\ell(y, \sigma(z)) = -y \log \sigma(z) - (1-y) \log(1-\sigma(z))$$

- Some example values:
  - $\ell\left(0, \frac{1}{2}\right) = \ell\left(1, \frac{1}{2}\right) = \log 2$
  - $\ell\left(1, \frac{9}{10}\right) \simeq .105$
  - $\ell\left(0, \frac{9}{10}\right) \simeq 2.303$
  - $\ell\left(1, \frac{99}{100}\right) \simeq 0.001$
  - $\ell\left(0, \frac{99}{100}\right) \simeq 4.605$



Predicted Probability of a "one"

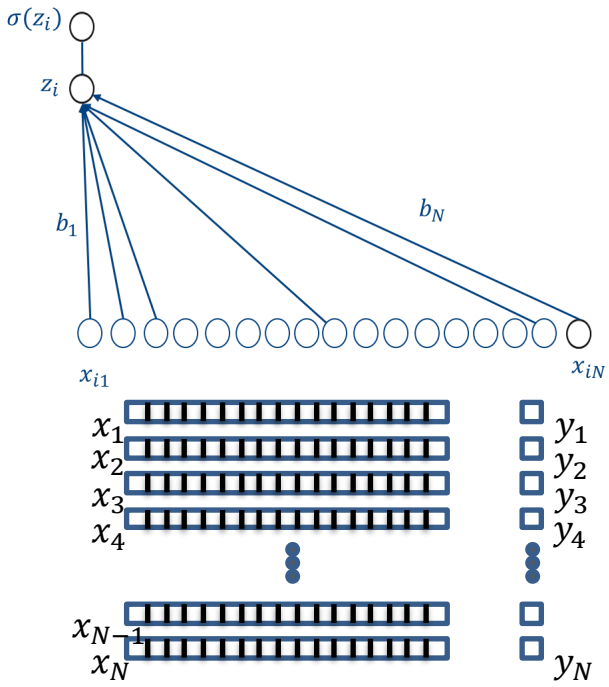
Optimization Goal for Binary Classification

- The optimization goal is to minimize the average loss

$$\boldsymbol{b}^* = \arg\min_{\boldsymbol{b}} \frac{1}{N} \sum_{i}^{N} \ell(y_i, \sigma(z_i))$$

- For binary (0/1) problems, the logistic or cross-entropy loss is

$$\ell\big(y, \sigma(z)\big) = -y \log \sigma(z) - (1-y) \log\big(1 - \sigma(z)\big)$$

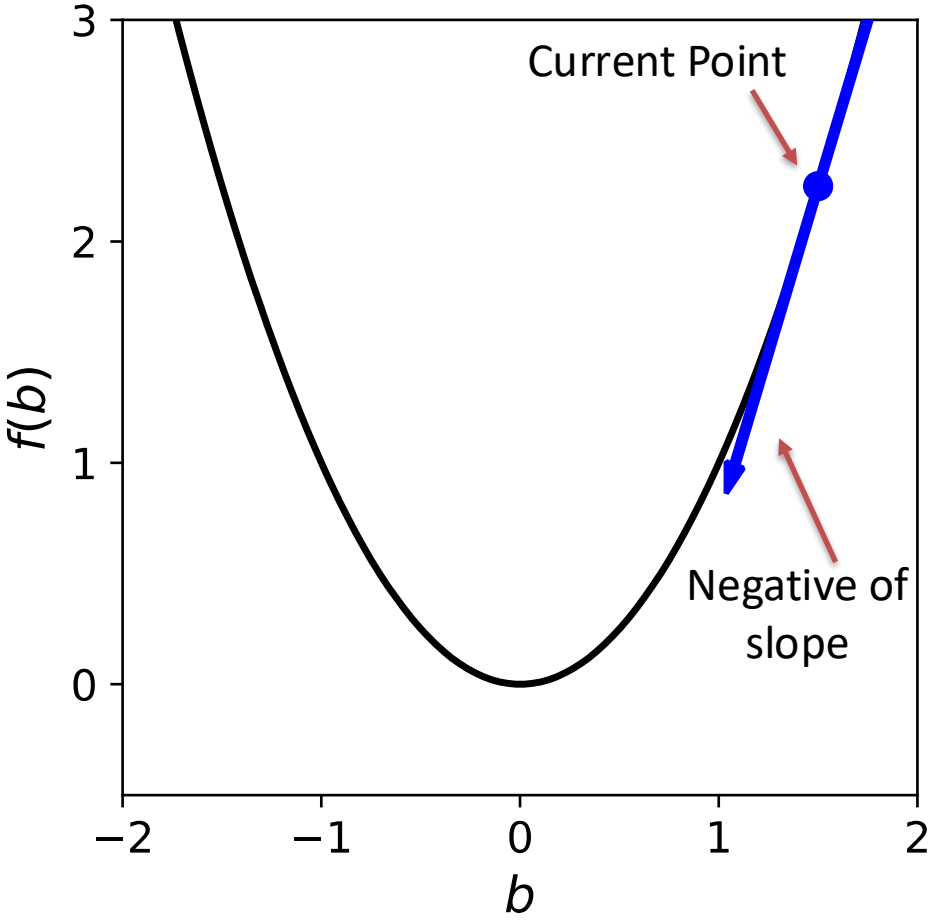


Brief Intro to Gradient Descent

LEARNING THE PARAMETERS

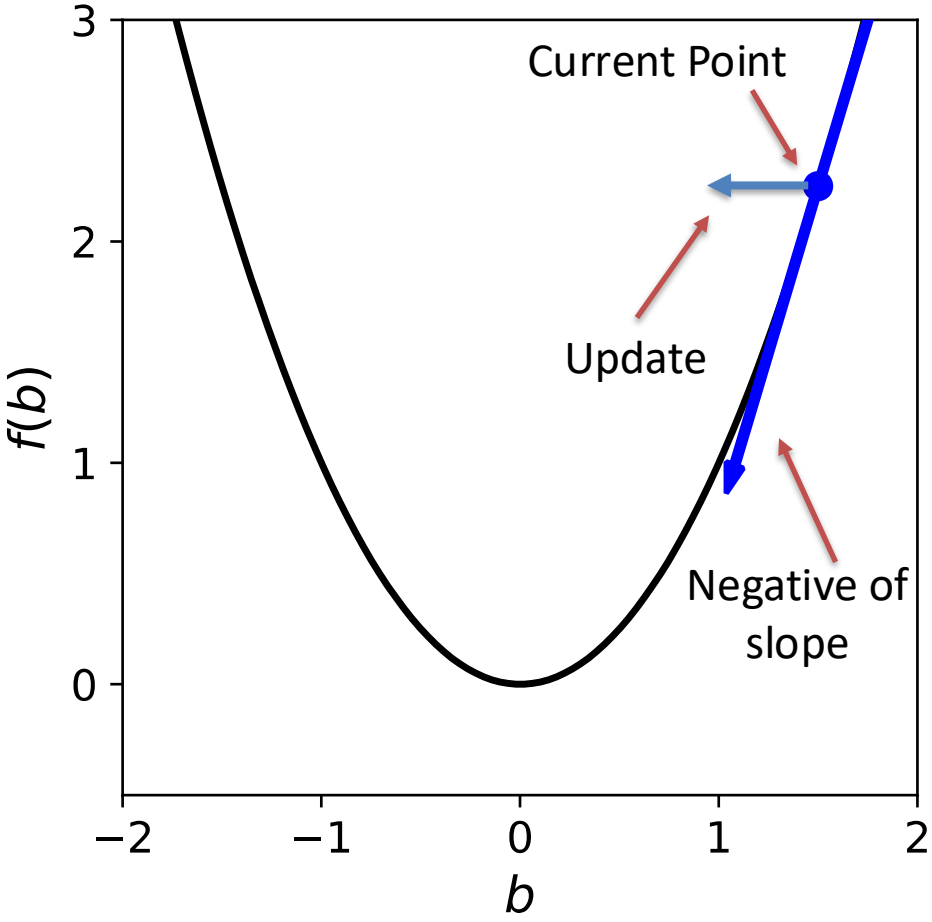
Visualization of Optimization Method

- We want to minimize a mathematical function (*i.e.*, our average loss function)
- One approach is to:
  1. Find the direction pointing "down the hill" (towards a smaller value)
  2. Move a bit in that direction
  3. Repeat 1-2 until satisfied



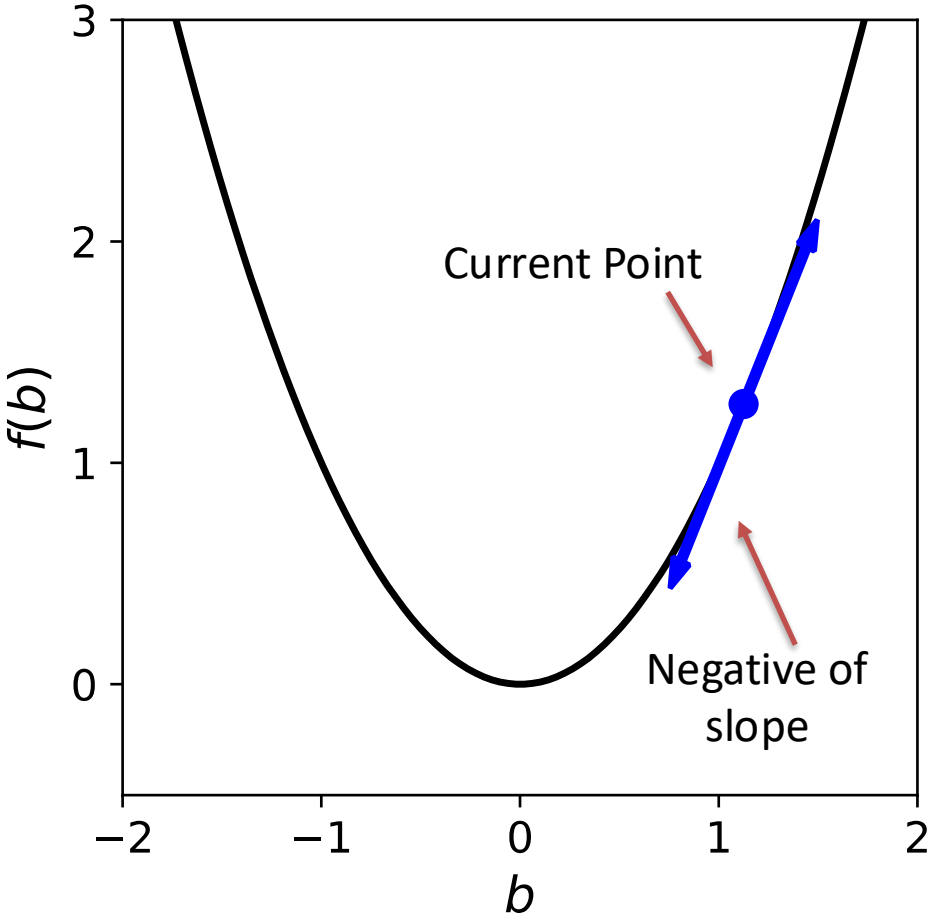
Visualization of Optimization Method

- We want to minimize a mathematical function (i.e. our average loss function)
- One approach is to:
  1. Find the direction pointing “down the hill” (towards a smaller value)
  2. Move a bit in that direction
  3. Repeat 1-2 until satisfied



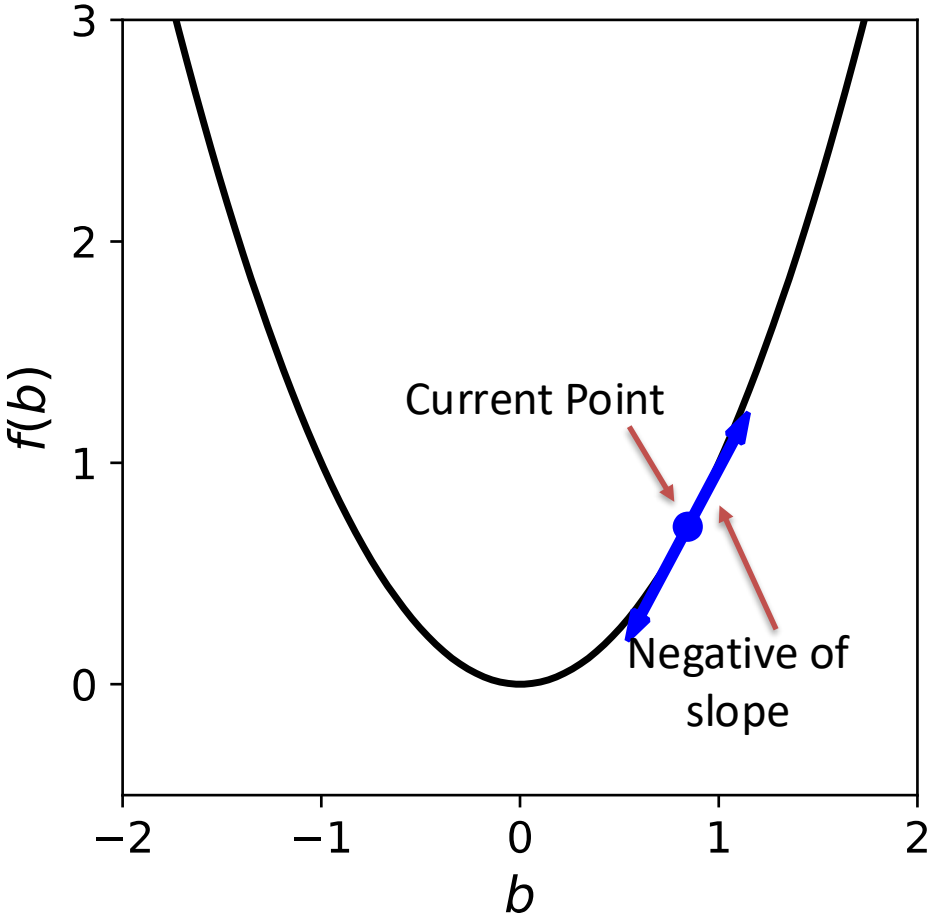
Visualization of Optimization Method

- We want to minimize a mathematical function (i.e. our average loss function)
- One approach is to:
  1. Find the direction pointing “down the hill” (towards a smaller value)
  2. Move a bit in that direction
  3. Repeat 1-2 until satisfied
- This shows the **first** update



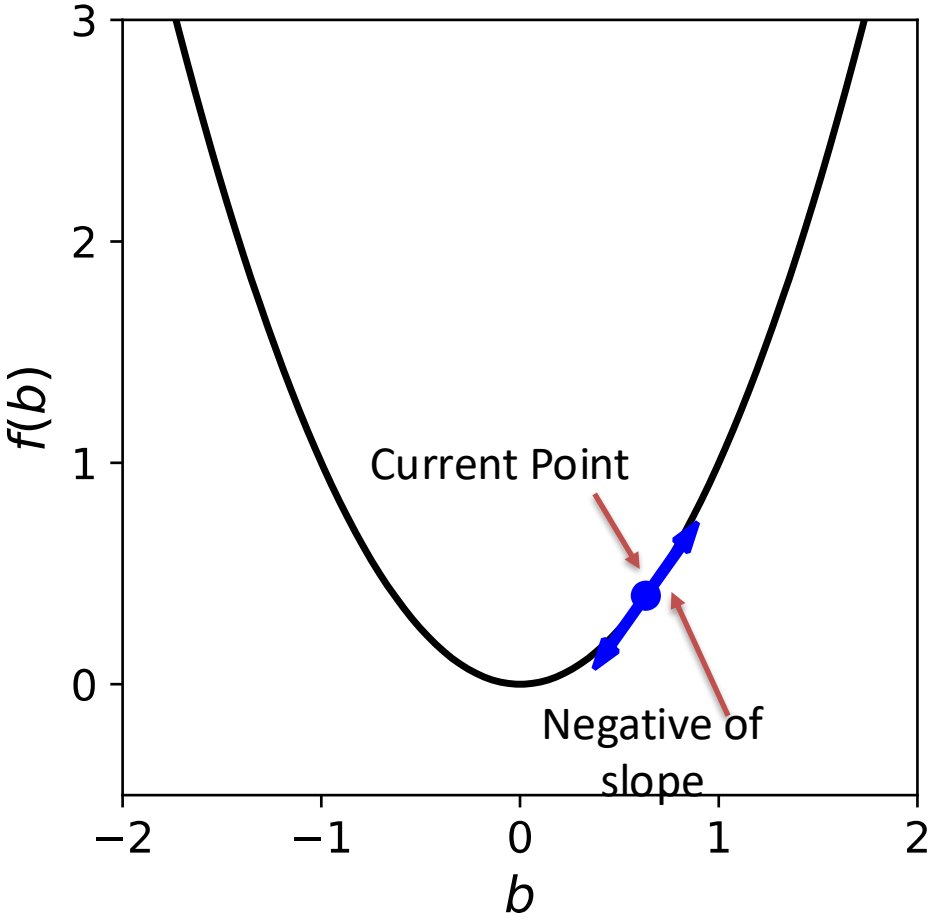
Visualization of Optimization Method

- We want to minimize a mathematical function (i.e. our average loss function)
- One approach is to:
  1. Find the direction pointing "down the hill" (towards a smaller value)
  2. Move a bit in that direction
  3. Repeat 1-2 until satisfied
- This shows the **second** update



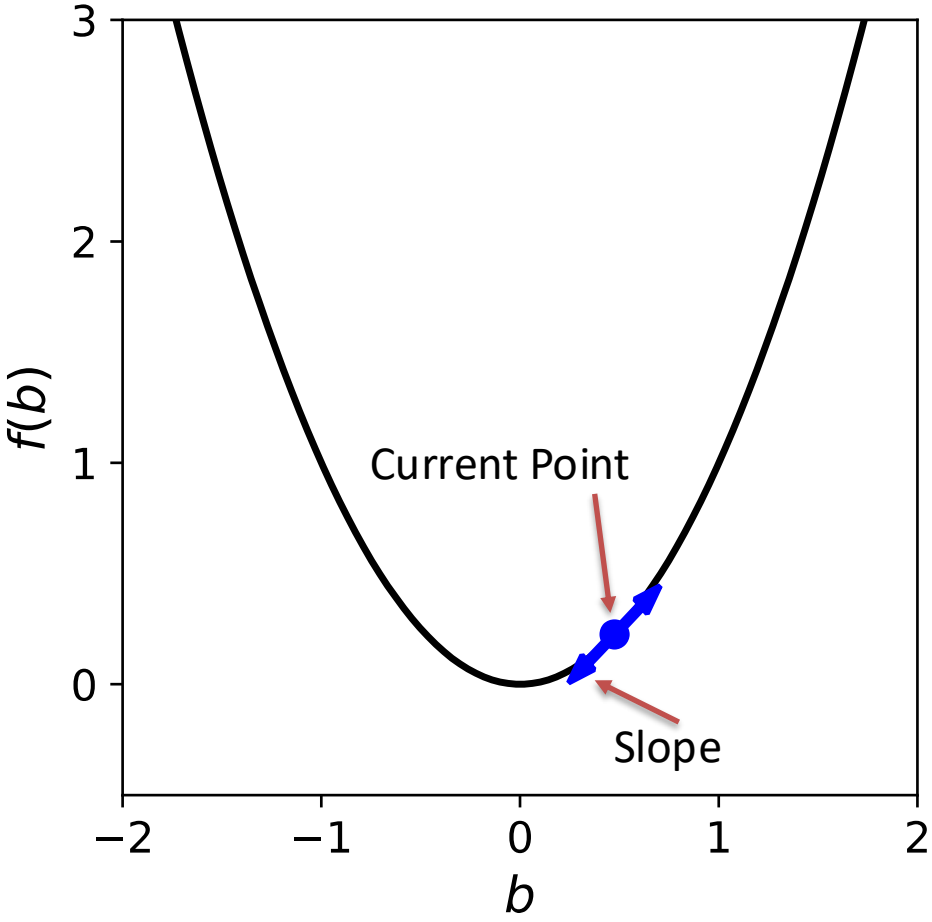
Visualization of Optimization Method

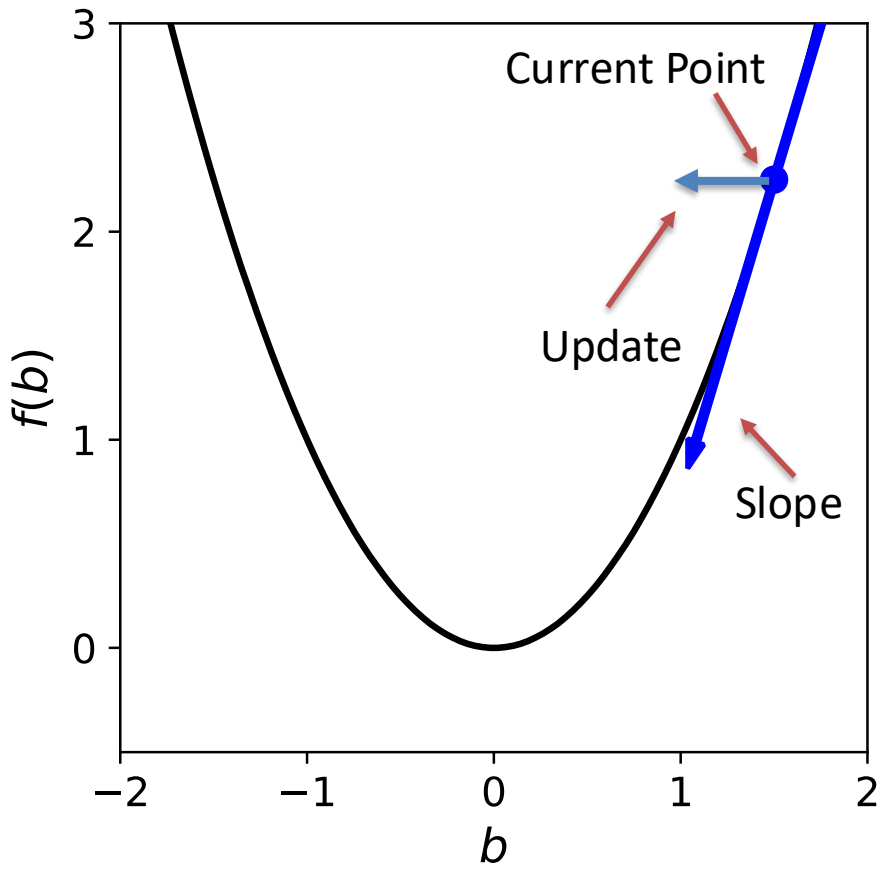
- We want to minimize a mathematical function (i.e. our average loss function)
- One approach is to:
  1. Find the direction pointing “down the hill” (towards a smaller value)
  2. Move a bit in that direction
  3. Repeat 1-2 until satisfied
- This shows the **third** update



Visualization of Optimization Method

- We want to minimize a mathematical function (i.e. our average loss function)
- One approach is to:
  1. Find the direction pointing “down the hill” (towards a smaller value)
  2. Move a bit in that direction
  3. Repeat 1-2 until satisfied
- This shows the **fourth** update





Mathematical Description of Gradient Descent

- We want to minimize a function
$$b^* = \arg\min_b f(b)$$
- Start at an initial value b^0
- We will run a series of updates to move from b^k to b^{k+1} (i.e., from b^0 to b^1)
- Iteratively run the procedure:
  1. Calculate the slope at the current point (For one parameter, this is the derivative. For multiple parameters, this is the *gradient*.): $\nabla f(b^k)$,
     ∇ means gradient or multidimensional slope
  2. Move in the direction of the negative gradient with *step size* α^k:
     $\mathrm{b}^{\mathrm{k}+1} = \mathrm{b}^{\mathrm{k}} - \alpha^k \nabla \mathrm{f}(\mathrm{b}^{\mathrm{k}})$
  3. Repeat 1-2 until converged

Brief Introduction to *Stochastic* Gradient Descent (SGD)

SCALING TO BIG DATA

Return to our Optimization Goal

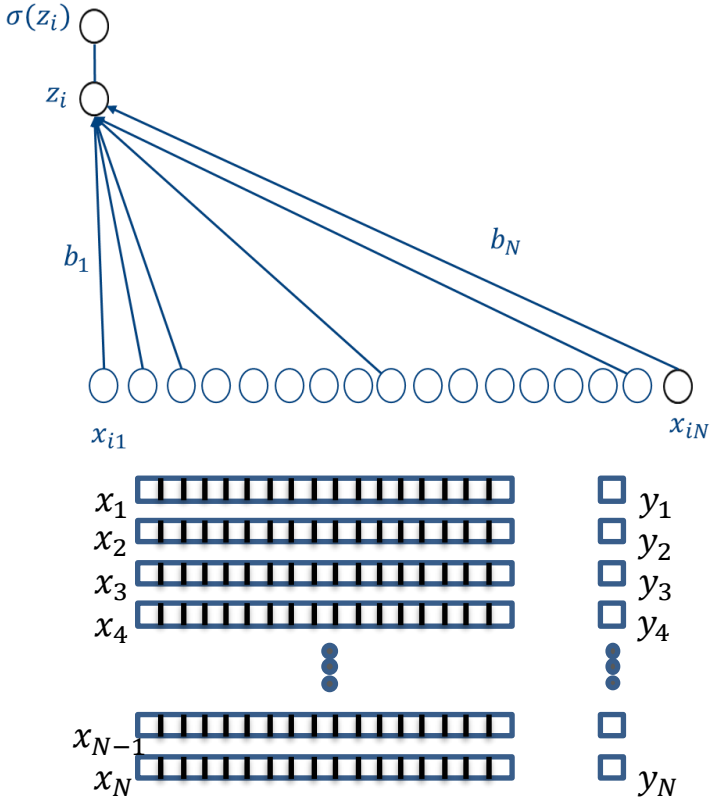
- We want to find the best parameters for all data points

$$\boldsymbol{b}^* = \arg\min_{\boldsymbol{b}} \frac{1}{N}\sum_{i}^{N} \ell(y_i, \sigma(z_i))$$

- However, calculating the gradient requires looking at *every* data point

$$\nabla \frac{1}{N}\sum_{i}^{N} \ell(y_i, \sigma(z_i)) = \frac{1}{N}\sum_{i}^{N} \nabla \ell(y_i, \sigma(z_i))$$

- This can be problematic in big data:
  - MNIST has ~60,000 images
  - ImageNet has ~1,000,000 images



Let's Just Approximate the Gradient

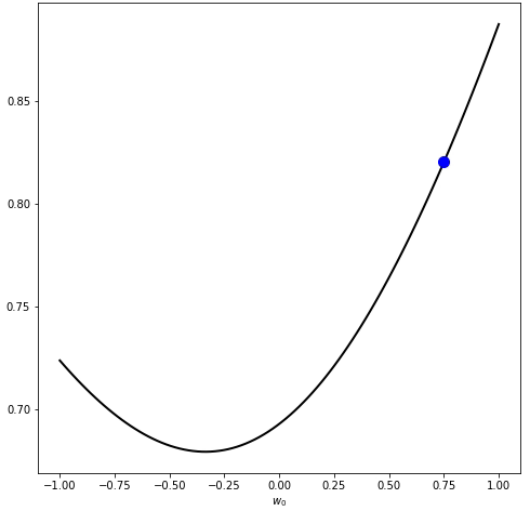
- We don't want to look at *every* data point to update our parameters
- Let's just take a single example j and use it to approximate the gradient

$$\nabla \ell\left(y_j, \sigma\left(z_j\right)\right) \simeq \frac{1}{N} \sum_{i=1}^{N} \nabla \ell(y_i, \sigma(z_i))$$

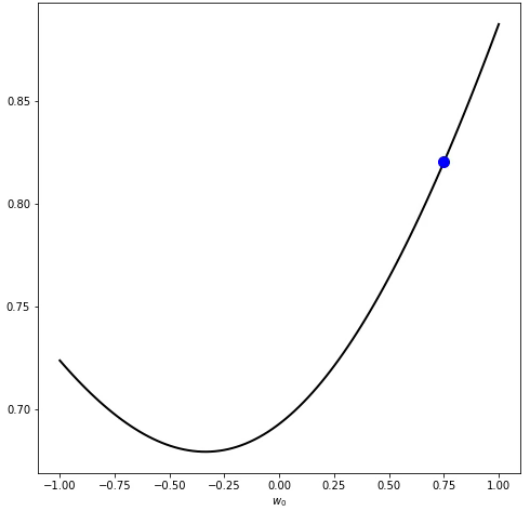
- In ImageNet, this is *1,000,000 times faster*
- Does this work? What would this look like?

Visualizations

Gradient Descent



Stochastic Gradient Descent



Data is Often Redundant

60,000 images, but really only 10 different types…

Mathematical Descriptions

Gradient Descent

1. Start with an initial $\boldsymbol{b}^0$
2. Calculate gradient $\nabla f(\boldsymbol{b}^k)$ over *all* data
3. Iteratively update: $\boldsymbol{b}^{k+1} \leftarrow \boldsymbol{b}^k - \alpha_k \nabla f(\boldsymbol{b}^k)$
4. Repeat 2-3 until solution is good enough

Stochastic Gradient Descent

1. Start with an initial $\boldsymbol{b}^0$
2. Choose a random data entry j
3. Estimate gradient $\widehat{\nabla f}\ (\boldsymbol{b}^k)$ by data point j
4. Iteratively update: $\boldsymbol{b}^{k+1} \leftarrow \boldsymbol{b}^k - \alpha_k \widehat{\nabla f}(\boldsymbol{b}^k)$
5. Repeat 2-4 until solution is good enough